Introduction to Artificial intelligence

Tutorial Sheet 3 (Searching) - Solution

Exercise 1

Give a complete problem formulation for each of the following. Choose a formulation that is precise enough to be implemented.

a. Using only four colors, you have to color a planar map in such a way that no two adjacent regions have the same color.

Four-Color Map Problem:

- **State Space:** The state space consists of all possible colorings of the regions on the planar map, where each region is assigned one of four colors.
- **Initial State:** The initial state is an uncolored planar map.
- **Goal State:** The goal state is reached when all regions on the map are colored such that no two adjacent regions have the same color.
- **Actions:** The agent can choose to color a region with one of the four available colors.
- **Transition Model:** The transition model updates the state by coloring a region with a chosen color.
- **Cost Function:** The cost function is the number of regions colored.

b. A 3-foot-tall (0.91 m) monkey is in a room where some bananas are suspended (hanging) from the 8-foot (2.44 m) ceiling. It would like to get the bananas. The room contains two stackable, movable, climbable 3-foot-high (0.91 m) crates (containers made of wood).





Monkey and Bananas Problem:

- **State Space:** The state space consists of the position of the monkey, the positions of the crates, and whether the monkey has the bananas or not (there is an infinite number of states).
- **Initial State:** The initial state is defined by the position of the monkey, the positions of the crates, and the fact that the monkey does not have the bananas.
- **Goal State:** The goal state is reached when the monkey has successfully obtained the bananas.
- **Actions:**
  - Hop on crate: The monkey hops onto one of the crates.
  - Hop off crate: The monkey hops off from the crate it's standing on.
  - Push crate from one spot to another: The monkey pushes one of the crates to a different position in the room.
  - Walk from one spot to another: The monkey walks from one spot to another within the room.
  - Grab bananas (if standing on crate): If the monkey is standing on a crate directly under the bananas, it can grab them.
- **Transition Model:** The transition model updates the state based on the chosen action, modifying the position of the monkey and/or crates accordingly.
- **Cost Function:** The cost function is the number of actions taken by the monkey to reach the goal state, i.e., to obtain the bananas.

c. You have a program that outputs the message "illegal input record" when fed a certain file of input records. You know that the processing of each record is independent of the processing of the other records. You may process just one part of the file (records) at a time. You want to discover what record is illegal as quickly as possible.

Problem Formulation:

- **State Space:** The state space consists of all possible combinations of indexes i and j that define subsets of input records (starting from record at position i to record at position j).
- **Initial State:** The initial state is defined as (0, n), where n is the total number of input records. This represents the entire set of input records.

- **Goal State:** The goal state is reached when the program outputs "illegal input record" for one specific record. This is represented as (i, j) where i = j, indicating a single record.
- **Actions:**
  - The agent can choose an action to test a subset of input records defined by indexes i and j.
  - An action is represented as "run the program for records between index i and j."
  - The agent can choose any valid combination of indexes i and j, where $0 \leq i \leq j \leq n$.
- **Transition Model:** run again on the first half of the records; run again on the second half of the records.
- **Cost Function:** The cost function is defined as the number of actions taken by the agent.

Note: This is a **contingency problem**; you need to see whether a run gives an error message or not to decide what to do next

d. You have three jugs (liquid containers), measuring 12 gallons[1], 8 gallons, and 3 gallons, and a water faucet[2]. None of the jugs has any measuring markings on it. You can fill the jugs up, or empty them out from one to another or onto the ground. You need to measure out exactly one gallon. How can you get exactly 1 gallon in one of the jugs?
Let a state [x,y,z] mean that there are x gallons in jug 1 (the one whose capacity is 12 gallons), y gallons in the second (8-gallon) jug and y gallons in the third (3-gallon) jug.

Initial state: jugs have values [0,0,0].
Successor function:
The actions are

- Fill(n) which fill jug n (n being 1, 2, or 3)
- Empty(n) and
- Pour(m,n): pour water from jug m to jug n (if n= 4 this means pour from jug m onto the ground)

This means that given values [x,y,z],

- generate [12,y,z], [x,8,z], [x,y,3] (by filling);
- [0,y,z], [x,0,z], [x,y,0] (by emptying); or
- for any two jugs with current values x and y, pour y into x means changing the jug with x to min(x + y, capacity of jug x), and decrements the jug with y by the amount gained by the first jug.

Cost function: Number of actions
Water Jug Problem:

- **State Space:** The state space consists of the amount of water in each jug.
- **Initial State:** The initial state is [0, 0, 0], where all jugs are empty.
- **Goal State:** The goal state is reached when one of the jugs contains exactly one gallon of water.
- **Actions:**
  - Fill the 12-gallon jug.
  - Fill the 8-gallon jug.
  - Fill the 3-gallon jug.
  - Empty the 12-gallon jug.
  - Empty the 8-gallon jug.
  - Empty the 3-gallon jug.
  - Pour water from one jug to another, changing the state as follows:
    - If pouring from jug X to jug Y:
      - The amount of water in jug X becomes the minimum of (amount of water in jug X + amount of water in jug Y) and the capacity of jug X.
      - The amount of water in jug Y decreases by the amount poured from jug X.
- **Transition Model:** The transition model updates the state based on the chosen action, modifying the amount of water in the jugs accordingly.
  You need to formally describe the results of the different actions. For instance,
  - Fill12([x,y,z]) ➔ [12,y,z], and the other Fill actions etc.
  - Empty12([x,y,z]) ➔ [0,y,z] and the other Empty actions etc.

[1] A gallon is approximately 3.785 liters.
[2] A water faucet is a device that allows you to turn the flow of water on and off.

- Pour(1,2,[x,y,z]) ➔ [min(x+y,capacity(1)), y-(x-min(x+y,capacity(1))), z]
- Do the same with the other Pour actions (1 to 3, 2 to 3, 3 to 1, 3 to 2, 2 to 1)

- **Cost Function:** The cost function is the number of actions taken to reach the goal state.

Exercise 2

Consider a state space where the start state is number 1 and each state k has two successors: numbers 2k and 2k + 1.

a. Draw the portion of the state space for states 1 to 15.

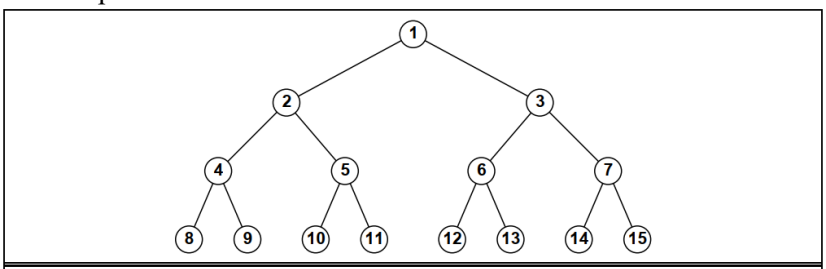


Figure S3.1 The state space for the problem defined in Ex. 3.15.

b. Suppose the goal state is 11. List the order in which nodes will be visited for breadth first search, depth-limited search with limit 3, and iterative deepening search.

Breadth-first: 1 2 3 4 5 6 7 8 9 10 11

Depth-limited: 1 2 4 8 9 5 10 11

Iterative deepening: depth 0: 1 depth 1: 1 2 3 depth 2: 1 2 4 5 3 6 7 depth 3: 1 2 4 8 9 5 10 11

c. How well would bidirectional search work on this problem? What is the branching factor in each direction of the bidirectional search?

Bidirectional search is very useful, because the only successor of n in the reverse direction is ⌊(n/2)⌋. This helps focus the search. The branching factor is 2 in the forward direction; 1 in the reverse direction.

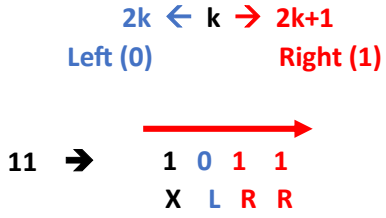
For example, when searching for the goal state 11, the forward Breadth first search would expand states 1, **2**, 3, and the backward search would expand states 11, 5, **2**. The two search frontiers would meet at state 2, which is the first node they have in common. The path is then detected as 1,2,5,11.

d. Does the answer to (c) suggest a reformulation of the problem that would allow you to solve the problem of getting from state 1 to a given goal state with almost no search?

Yes, the problem could be reformulated as finding the path from 11 to 1 instead of from 1 to 11. This would allow us to start the search from the goal state (11) and explore its predecessors, which is more efficient than searching from the start state (1) to the goal state (11).

e. Call the action going from k to 2k "Left", and the action going from k to 2k + 1 "Right". Can you find an algorithm that outputs the solution to this problem without any search at all?

The solution can be read off the binary numeral for the goal number. Write the goal number in binary. Since we can only reach positive integers, this binary expansion begins with a 1. From most- to least- significant bit, skipping the initial 1, go Left to the node 2n if this bit is 0 and go Right to node 2n + 1 if it is 1. For example, suppose the goal is 11, which is 1011 in binary. The solution is therefore Left, Right, Right.



Exercise 3

A basic wooden railway set contains the pieces shown in Figure 3.32. The task is to connect these pieces into a railway that has no overlapping tracks and no loose ends where a train could run off onto the floor.

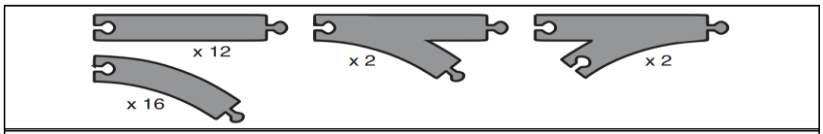


Figure 3.32 The track pieces in a wooden railway set; each is labeled with the number of copies in the set. Note that curved pieces and "fork" pieces ("switches" or "points") can be flipped over so they can curve in either direction. Each curve subtends 45 degrees.

a. Suppose that the pieces fit together *exactly* with no slack. Give a precise formulation of the task as a search problem.

- **State Space:** The state space consists of all possible arrangements of the railway pieces.
- **Initial State:** An empty railway track or one arbitrarily selected piece.
- **Goal State:** all pieces used in a single connected track, no open pegs or holes, no overlapping tracks.
- **Actions:** The agent can choose to add a railway piece to the current layout, ensuring it fits properly without overlapping tracks and without creating loose ends.
- **Successor function:** for any open peg, add any piece type from remaining types. (You can add to open holes as well, but that isn't necessary as all complete tracks can be made by adding to pegs.) For a curved piece, add in either orientation; for a fork, add in either orientation and (if there are two holes) connecting at either hole. It's a good idea to disallow any overlapping configuration, as this terminates hopeless configurations early. (Note: there is no need to consider open holes, because in any solution these will be filled by pieces added to open pegs.)
- **Cost Function:** The cost function is the number of actions taken to reach the goal state.

b. Identify a suitable uninformed search algorithm for this task and explain your choice.
All solutions are at the same depth, so depth-first search would be appropriate. The space is very large, so uniform-cost and breadth-first would fail, and iterative deepening simply does unnecessary extra work. There are many repeated states, so it might be good to use a closed list.

c. Explain why removing any one of the "fork" pieces makes the problem unsolvable.
A solution has no open pegs or holes, so every peg is in a hole, so there must be equal numbers of pegs and holes. Removing a fork violates this property. There are two other "proofs" that are acceptable: 1) a similar argument to the effect that there must be an even number of "ends"; 2) each fork creates two tracks, and only a fork can rejoin those tracks into one, so if a fork is missing it won't work. The argument using pegs and holes is actually more general, because it also applies to the case of a three-way fork that has one hole and three pegs or one peg and three holes. The "ends" argument fails here, as does the fork/rejoin argument (which is a bit handwavy anyway).

d. Give an upper bound on the total size of the state space defined by your formulation. (*Hint*: think about the maximum branching factor for the construction process and the maximum depth, ignoring the problem of overlapping pieces and loose ends. Begin by pretending that every piece is unique.)
Pretending each piece is unique, any piece can be added to a peg, giving at most $12 + (2 \cdot 16) + (2 \cdot 2) + (2 \cdot 2 \cdot 2) = 56$ choices per peg. The total depth is 32 (there are 32 pieces), so an upper bound is $56^{32}/(12! \cdot 16! \cdot 2! \cdot 2!)$ where the factorials deal with permutations of identical pieces. One could do a more refined analysis to handle the fact that the branching factor shrinks as we go down the tree, but it is not pretty.

Exercise 4
Describe a state space in which iterative deepening search performs much worse than depth-first search (for example, $O(n^2)$ vs. $O(n)$).
Consider a domain in which every state has a single successor, and there is a single goal at depth n. Then depth-first search will find the goal in n steps, whereas iterative deepening search will take $1 + 2 + 3 + \cdots + n = O(n^2)$ steps.

Exercise 5
Consider the unbounded version of the regular 2D grid shown in Figure 3.9. The start state is at the origin, (0,0), and the goal state is at (x, y).

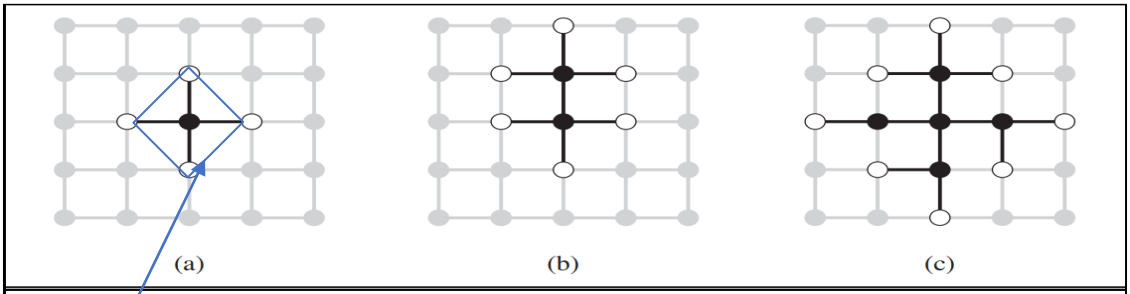


Figure 3.9 The separation property of GRAPH-SEARCH, illustrated on a rectangular-grid problem. The frontier (white nodes) always separates the explored region of the state space (black nodes) from the unexplored region (gray nodes). In (a), just the root has been expanded. In (b), one leaf node has been expanded. In (c), the remaining successors of the root have been expanded in clockwise order.

a. What is the branching factor b in this state space?
The branching factor is 4 (number of neighbors of each location).

b. How many distinct states are there on the frontier at depth k (for k > 0)?
4 at depth one, 8 at depth 2, 12 at depth 3, …., 4k at depth k. The states at depth k form a square rotated at 45 degrees to the grid.

c. How many distinct states are there in total whether explored or on the frontier at depth k (for k > 0)?
At depth k = 1: g(1) = 4 (on the frontier) + 1 (explored) = 5 ➔ 4k +1
At depth k = 2: g(2) = 8 (on the frontier) + 5 (explored) = 13 ➔ 4k + g(2)
One can see that the general recursive expression is
g(k) = 4k + g(k-1)
with g(0) = 1

d. What is the maximum number of nodes expanded by breadth-first tree search?
BFS expends exponentially many nodes: counting precisely: 1 at depth 0, 4 at depth 1, 16 at depth 2, 64 at depth 3, 4^k at depth k. It is a geometric series with first term = 4, common ratio = 4 and the number of terms at depth k = k+1, we get $(4^{k+1} - 1)/3$, (because it is a geometric series the sum is $a * (r^n - 1) / (r - 1)$ where $a=1$, $r=4$, and $n = k+1$).
any point at depth k have x+y=k (x and y are both in absolute value, the points at k are all the points in the square rotated at 45°).

e. What is the maximum number of nodes expanded by breadth-first graph search?
Here states are visited only once so we will sum the number of states in all the squares at each depth: 1 + 4 + 8 + 12 … + 4k. The sum (ignoring the root for now we add it later) is an arithmetic series with first term (nodes at depth 1) = 4*1=4, and the last term (nodes at depth k) = 4*k, the number of terms is k (we ignored the root). The sum will be: k/2(4+4k) = 2k(1+k), we add the one of the root that we ignored before: 2k(1+k)+1. We replace depth k by x+y and get: 2(x + y)(x + y + 1) + 1.

f. Is h = |u − x| + |v − y| an admissible heuristic for a state at (u, v)? Explain.
True; this is the Manhattan distance metric which gives the optimal number of steps to go from (u,v) to (x,y).

g. How many nodes are expanded by A∗ graph search using h?
Exactly x+y.

h. Does h remain admissible if some links are removed?
True; removing links may induce detours, which require more steps, so h is an underestimate.

i. Does h remain admissible if some links are added between nonadjacent states?
False; this can reduce the actual path length below the Manhattan distance meaning we can get from (u,v) to (x,y) in less than x+y.

Exercise 6

Suppose two friends live in different cities on a map, such as the Romania map shown in Figure 3.2.

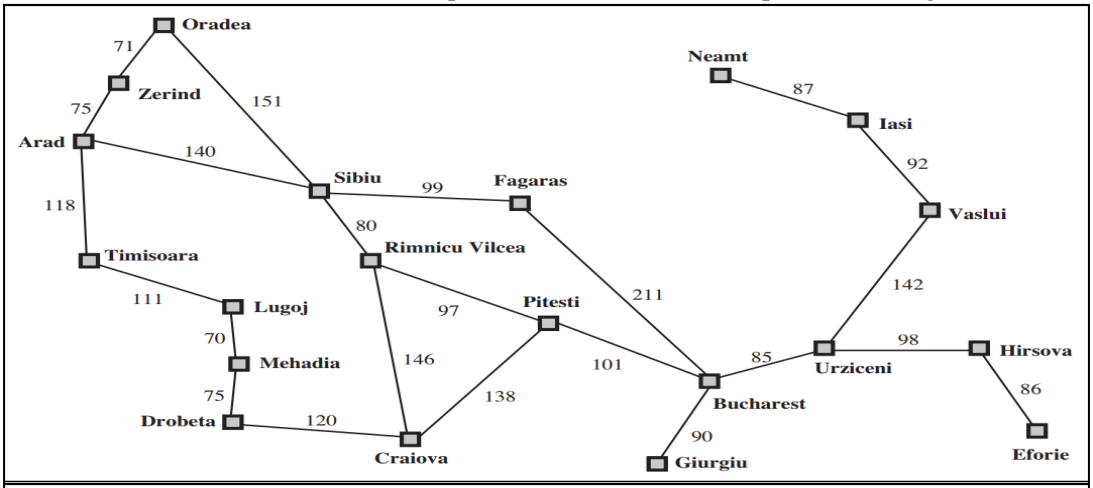


Figure 3.2 A simplified road map of part of Romania.

On every turn, we can simultaneously move each friend to a neighboring city on the map. The amount of time needed to move from city i to neighbor j is equal to the road distance d(i, j) between the cities, but on each turn the friend that arrives first must wait until the other one arrives (and calls the first on his/her cell phone) before the next turn can begin. We want the two friends to meet as quickly as possible.

a. Write a detailed formulation for this search problem. (You will find it helpful to define some formal notation here.)
State space: States are all possible city pairs (i,j). The map is *not* the state space.
Successor function: The successors of (i,j) are all pairs (x,y) such that Adjacent(x,i) and Adjacent(y,j).
Goal: Be at (i,i) for some i.
Step cost function: The cost to go from (i,j) to (x,y) is max(d(i,x),d(j,y)).

b. Let D(i, j) be the straight-line distance between cities i and j. Which of the following heuristic functions are admissible?

(i) D(i, j);
(ii) 2 * D(i, j);
(iii) D(i, j)/2.

In the best case, the friends head straight for each other in steps of equal size, reducing their separation by twice the time cost on each step. Hence (iii) is admissible.

c. Are there completely connected maps for which no solution exists?
yes: e.g., a map with two nodes connected by one link. The two friends will swap places forever. The same will happen on any chain if they start an odd number of steps apart. (One can see this best on the graph that represents the state space, which has two disjoint sets of nodes.) The same even holds for a grid of any size or shape, because every move changes the Manhattan distance between the two friends by 0 or 2.

d. Are there maps in which all solutions require one friend to visit the same city twice?
yes: take any of the unsolvable maps from part (c) and add a self-loop to any one of the nodes. If the friends start an odd number of steps apart, a move in which one of the friends takes the self-loop changes the distance by 1, rendering the problem solvable. If the self-loop is not taken, the argument from (c) applies and no solution is possible.

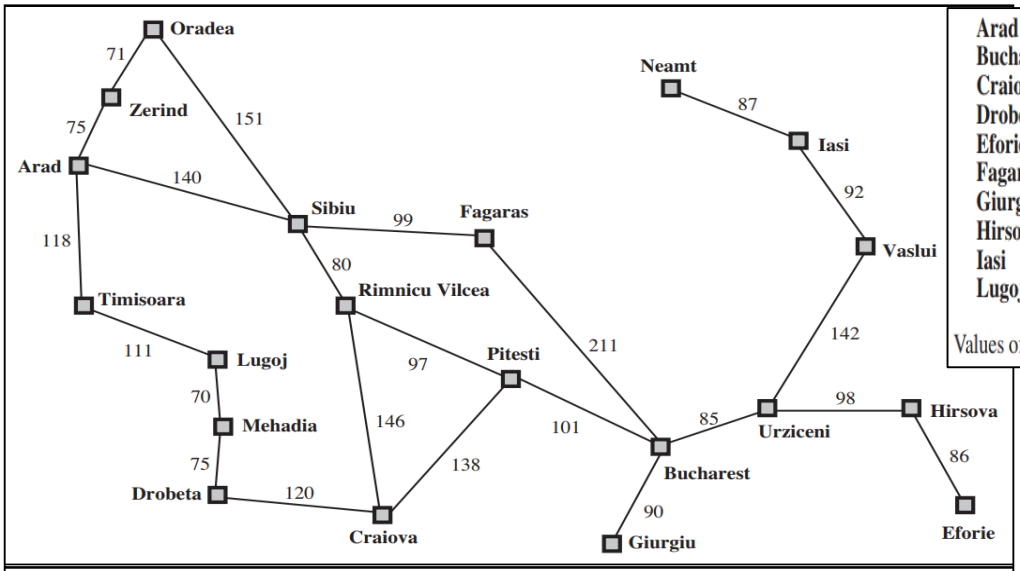
Exercise 7
Which of the following are true and which are false? Explain your answers.
a. Depth-first search always expands at least as many nodes as A∗ search with an admissible heuristic.
False: a lucky DFS might expand exactly d nodes to reach the goal. A∗ largely dominates any graph-search algorithm that is *guaranteed to find optimal solutions*.
b. h(n) = 0 is an admissible heuristic for the 8-puzzle.
True: h(n) = 0 is always an admissible heuristic, since costs are nonnegative.
c. A∗ is of no use in robotics because percepts, states, and actions are continuous.
True: A* search is often used in robotics; the space can be discretized or skeletonized.

d. Breadth-first search is complete even if zero step costs are allowed.

True: depth of the solution matters for breadth-first search, not cost.

e. Assume that a rook can move on a chessboard any number of squares in a straight line, vertically or horizontally, but cannot jump over other pieces. Manhattan distance is an admissible heuristic for the problem of moving the rook from square A to square B in the smallest number of moves.
False: a rook can move across the board in move one, although the Manhattan distance from start to finish is 8.

Exercise 8
Trace the operation of A∗ search applied to the problem of getting to Bucharest from Lugoj using the straight-line distance heuristic. That is, show the sequence of nodes that the algorithm will consider and the f, g, and h score for each node.



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Values of h_{SLD}—straight-line distances to Bucharest.

Figure 3.2 A simplified road map of part of Romania.

The sequence of queues is as follows, order by priority (for graph search):

- L [0+244=244]
- M [70+241=311], T [111+329=440]
- D [145+242=387], T [111+329=440]
- C [265+160=425], T [111+329=440]
- T [111+329=440], R [411+193=604], P[403+100=503]
- P [403+100=503], R[411+193=604], A[229+366=595]
- B [504+0=504], R[411+193=604], A[229+366=595]

The sequence of queues is as follows, order by priority (for tree search):

- L[0+244=244]
- M[70+241=311], T[111+329=440]
- L[140+244=384], D[145+242=387], T[111+329=440]
- D[145+242=387], T[111+329=440], M[210+241=451], T[251+329=580]
- C[265+160=425], T[111+329=440], M[210+241=451], M[220+241=461], T[251+329=580]
- T[111+329=440], M[210+241=451], M[220+241=461], P[403+100=503], T[251+329=580], R[411+193=604], D[385+242=627]
- M[210+241=451], M[220+241=461], L[222+244=466], P[403+100=503], T[251+329=580], A[229+366=595], R[411+193=604], D[385+242=627]
- M[220+241=461], L[222+244=466], P[403+100=503], L[280+244=524], D[285+242=527], T[251+329=580], A[229+366=595], R[411+193=604], D[385+242=627]
- L[222+244=466], P[403+100=503], L[280+244=524], D[285+242=527], L[290+244=534], D[295+242=537],T[251+329=580], A[229+366=595], R[411+193=604], D[385+242=627]

- P[403+100=503], L[280+244=524], D[285+242=527], M[292+241=533], L[290+244=534], D[295+242=537], T[251+329=580], A[229+366=595], R[411+193=604], D[385+242=627], T[333+329=662]
- B[504+0=504], L[280+244=524], D[285+242=527], M[292+241=533], L[290+244=534], D[295+242=537], T[251+329=580], A[229+366=595], R[411+193=604], D[385+242=627], T[333+329=662], R[500+193=693], C[541+160=701]